

Nama : Muhammad Alfin Maulana

NPM : 5230311052

Catatan BAB III Fitur Pada Laravel

Oke catatan ini akan menjelaskan terkait dengan fitur-fitur yang ada di dalam Laravel itu sendiri. Dalam pengembangan aplikasi menggunakan Laravel, kita sering berinteraksi dengan **Artisan**. Artisan dapat dipahami sebagai antarmuka baris perintah (CLI) yang berfungsi sebagai asisten cerdas dalam kerangka kerja Laravel. Fungsinya adalah untuk mengotomatisasi berbagai tugas teknis yang berulang, sehingga pengembang tidak perlu melakukannya secara manual.

Untuk memanfaatkan fitur ini, perintah diketikkan melalui terminal atau *command prompt* di dalam direktori proyek. Beberapa kemampuan utama Artisan meliputi pembuatan kerangka file (seperti Controller, Model, Migration), manajemen basis data (migrasi dan *seeding*), serta menjalankan server pengembangan lokal.

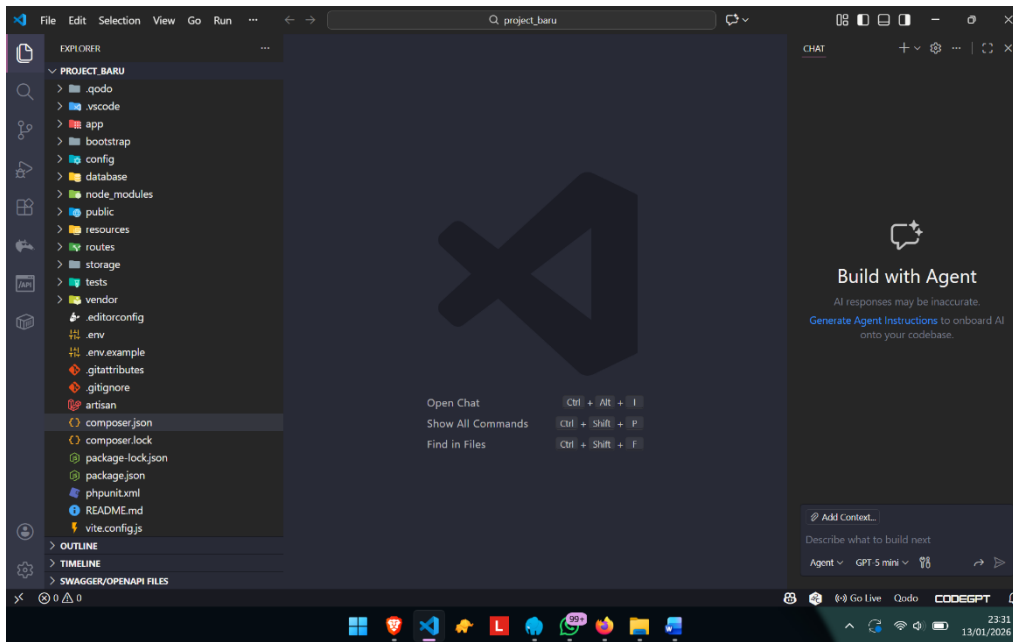
Berikut adalah tabel perintah umum yang sering digunakan:

Perintah	Deskripsi Fungsi
php artisan serve	Mengaktifkan server lokal agar aplikasi bisa diakses di browser (biasanya port 8000).
php artisan make:model NamaModel -m	Membuat file Model sekaligus file migrasi pendukungnya secara otomatis.
php artisan migrate	Menerapkan perubahan struktur tabel yang ada di kode ke dalam database MySQL.
php artisan db:seed	Mengisi tabel database dengan data awal atau data uji coba.
php artisan make:seeder NamaSeeder	Membuat file class seeder baru untuk mendefinisikan data yang akan diinput.
php artisan migrate:fresh --seed	Menghapus seluruh tabel, membuatnya ulang dari nol, dan langsung mengisinya dengan data dummy.

Selanjutnya saya akan mendemokan apa saja fitur yang ada di dalam Laravel ini.

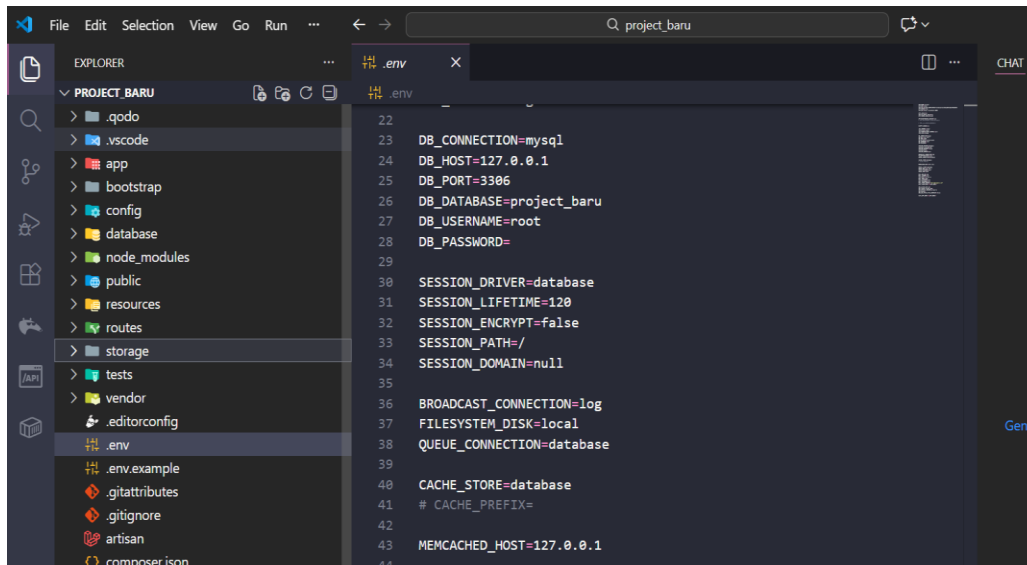
Persiapan Awal

- **Inisialisasi Proyek:** Langkah pertama adalah membuat proyek baru melalui terminal. Arahkan direktori ke C:\xampp\htdocs dan jalankan perintah `laravel new nama_project`. Proses ini akan mengunduh dependensi yang dibutuhkan, mungkin cara spesifik nya sudah di dokumentasikan dicatatan sebelumnya.



Oke project kita sudah dibuat dan siap untuk mencoba beberapa fitur di dalam Laravel tersebut.

- **Konfigurasi Database:** Setelah proyek terbentuk, buka file `.env` di editor kode. Sesuaikan konfigurasi koneksi basis data, terutama pada bagian `DB_DATABASE`. Di sini saya menggunakan nama database `bab3` yang sebelumnya telah dibuat kosong melalui `phpMyAdmin`.



The screenshot shows the Visual Studio Code editor with the `.env` file open. The file contains configuration for a Laravel application. The database configuration is as follows:

```
22 DB_CONNECTION=mysql
23 DB_HOST=127.0.0.1
24 DB_PORT=3306
25 DB_DATABASE=project_baru
26 DB_USERNAME=root
27 DB_PASSWORD=
28
29
30 SESSION_DRIVER=database
31 SESSION_LIFETIME=120
32 SESSION_ENCRYPT=false
33 SESSION_PATH=/
34 SESSION_DOMAIN=null
35
36 BROADCAST_CONNECTION=log
37 FILESYSTEM_DISK=local
38 QUEUE_CONNECTION=database
39
40 CACHE_STORE=database
41 # CACHE_PREFIX=
42
43 MEMCACHED_HOST=127.0.0.1
44
```

Disini saya membuat database dengan nama **project_baru** jadi code tersebut akan mengeneret database secara otomatis, dengan perintah **php artisan migrate**.

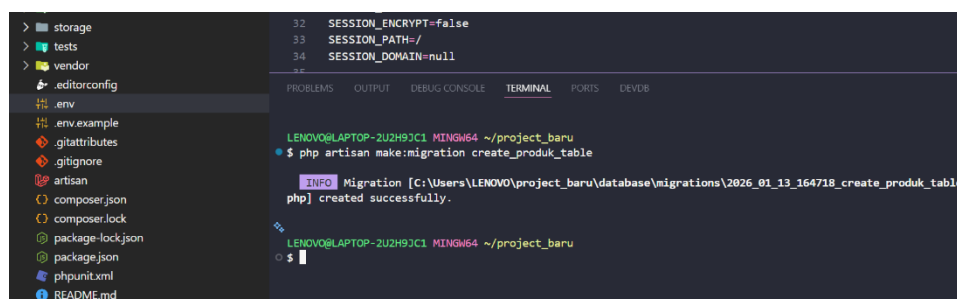
Mungkin selanjutnya kita akan mencoba fitur migration, untuk membuat kerangka kerja dari table di dalam database yang dibut sebelumnya.

- Membuat Migration

Penjelasan Konsep: Migration berperan seperti *version control* (pengendali versi) untuk struktur database. Fitur ini memungkinkan kita mendefinisikan tabel dan kolom menggunakan kode PHP, bukan SQL manual. Keuntungannya, seluruh tim pengembang memiliki struktur database yang seragam dan perubahan dapat dilacak atau dibatalkan dengan mudah.

Langkah-langakhnya seperti berikut :

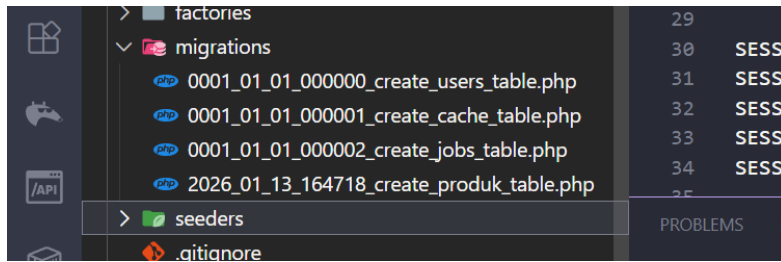
1. Buat file migrasi baru untuk tabel produk dengan perintah:



The screenshot shows the Visual Studio Code editor with the `.env` file open and the terminal window at the bottom. The terminal shows the command `php artisan make:migration create_produk_table` being executed, and the output is:

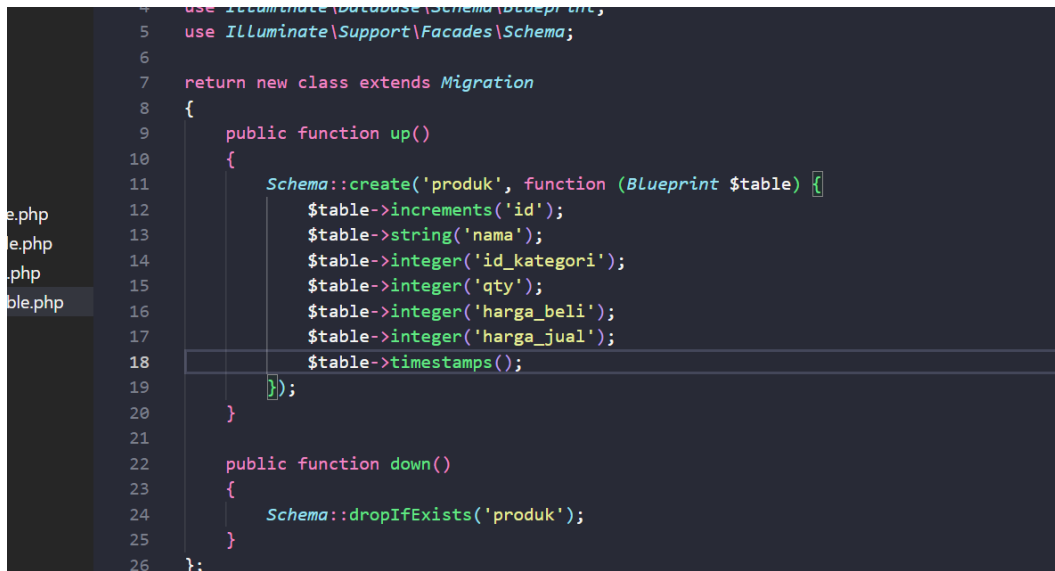
```
LENOVO@LAPTOP-2U2H93C1 MINGW64 ~/project_baru
$ php artisan make:migration create_produk_table
INFO Migration [C:\Users\LENOVO\project_baru\database\Migrations\2026_01_13_164718_create_produk_table.php] created successfully.
LENOVO@LAPTOP-2U2H93C1 MINGW64 ~/project_baru
$
```

Denagn perintah tersebut maka file migration sudah terbuat, kita bisa cek file tersebut, di dalam folder Database\Migrations ada file yang telah dibuat tadi.



Bisa kita lihat file yang tadi kita buat dengan perintah **php artisan make:migration create_produk_table** berhasil dibuat.

2. Buka file tersebut, lalu pada method up(), definisikan kolom-kolom yang dibutuhkan seperti nama, id_kategori, qty, harga_beli, dan harga_jual

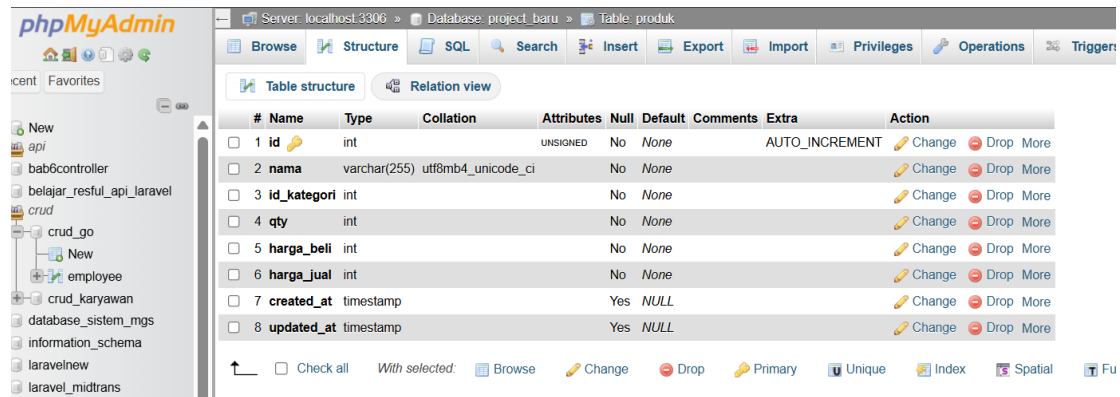


Okee selanjutnya kita tinggal menerapkan scema table tersebut dengan perintah seperti di bawah ini.

3. Terapkan skema tersebut ke database dengan perintah:



Perintah tersebut akan membuat schema table di dalam database yang sebelumnya kita buat di dalam file .env, kita cek langsung di dalam phpMyAdmin kita.



The screenshot shows the phpMyAdmin interface with the 'Table structure' tab selected for the 'table: produk' database. The table structure is as follows:

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	id	int		UNSIGNED	No	None		AUTO_INCREMENT	Change Drop More
2	nama	varchar(255)	utf8mb4_unicode_ci		No	None			Change Drop More
3	id_kategori	int			No	None			Change Drop More
4	qty	int			No	None			Change Drop More
5	harga_beli	int			No	None			Change Drop More
6	harga_jual	int			No	None			Change Drop More
7	created_at	timestamp			Yes	NULL			Change Drop More
8	updated_at	timestamp			Yes	NULL			Change Drop More

Oke jika sudah ada nama database dan tablenya, menandakan bahwa database dan table yang kita buat sudah berhasil dibuat.

- Fitur lain dalam migration yang dapat diterapkan
 Fitur ini bekerja dengan menjalankan metode `down()` yang terdapat pada file migrasi. Berikut adalah beberapa perintah Artisan yang digunakan untuk manajemen pembatalan migrasi:
 1. **Pembatalan Terakhir (*Last Batch*)** Perintah dasar yang digunakan untuk membatalkan satu kelompok migrasi (batch) yang terakhir kali dijalankan adalah:
`php artisan migrate:rollback`
 2. **Pembatalan Bertahap (*Step Rollback*)** Jika kita ingin membatalkan beberapa tahapan migrasi sekaligus (misalnya 2 kali migrasi terakhir), kita dapat menambahkan opsi `--step`. Contoh perintahnya adalah:
`php artisan migrate:rollback --step=2`
 3. **Reset Total (*Reset*)** Untuk membatalkan **seluruh** riwayat migrasi yang pernah dilakukan dan mengosongkan skema database (mengembalikan aplikasi ke kondisi awal), digunakan perintah:
`php artisan migrate:reset`
 4. **Refresh Database (*Refresh*)** Perintah ini sering digunakan saat tahap pengembangan awal (*development*). Perintah ini akan melakukan *rollback* total (menghapus semua tabel) dan kemudian langsung menjalankan perintah migrate kembali dari awal secara otomatis.
`php artisan migrate:refresh`
- Memodifikasi Tabel (Mengganti nama table)
 Pada tahap ini, saya perlu melakukan perubahan pada struktur database yang sudah ada. Karena penamaan tabel sebelumnya adalah `produk` (singular), saya ingin mengubahnya menjadi `barangs` (plural) agar lebih sesuai dengan standar penamaan di Laravel yang nantinya akan memudahkan integrasi dengan Model.

Berikut adalah langkah-langkah kerja yang saya lakukan:

1. Membuat File Migration Baru

Alih-alih mengedit file migrasi lama yang berisiko merusak histori database, saya memutuskan untuk membuat file migrasi baru khusus untuk perubahan nama ini. Saya menjalankan perintah berikut di terminal:

```
LENOVO@LAPTOP-2U2H9JC1 MINGW64 ~/project_baru
● $ php artisan make:migration ubah_tabel_produk

[INFO] Migration [C:\Users\LENOVO\project_baru\database\Migrations\2026_01_13_200331_ubah_tabel_produk.php] created successfully.

LENOVO@LAPTOP-2U2H9JC1 MINGW64 ~/project_baru
○ $
```

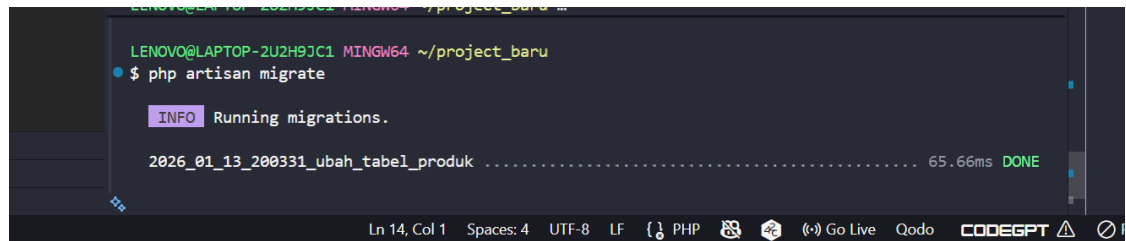
2. Menuliskan Logika Perubahan (Scripting)

Selanjutnya, saya membuka file migrasi yang baru saja terbentuk tersebut di VSCode. Di sini, saya memodifikasi dua metode utama, yaitu `up()` dan `down()`, untuk memberi instruksi kepada Laravel tentang apa yang harus diubah.

```
... 2026_01_13_164718_create_produk_table.php 2026_01_13_200331_ubah_tabel_produk.php X
database > migrations > 2026_01_13_200331_ubah_tabel_produk.php
5 use Illuminate\Support\Facades\Schema;
6
7 return new class extends Migration
8 {
9     /**
10      * Run the migrations.
11      */
12     public function up(): void
13     {
14         if (Schema::hasTable('produk')) {
15             Schema::rename('produk', 'barangs');
16         }
17     }
18
19     /**
20      * Reverse the migrations.
21      */
22     public function down(): void
23     {
24         if (Schema::hasTable('barangs')) {
25             Schema::rename('barangs', 'produk');
26         }
27     }
28 };
29
```

3. Mengeksekusi Migrasi

Setelah yakin kodenya benar, saya menyimpan file tersebut dan kembali ke terminal. Saya menjalankan perintah migrasi untuk menerapkan perubahan tersebut ke database:



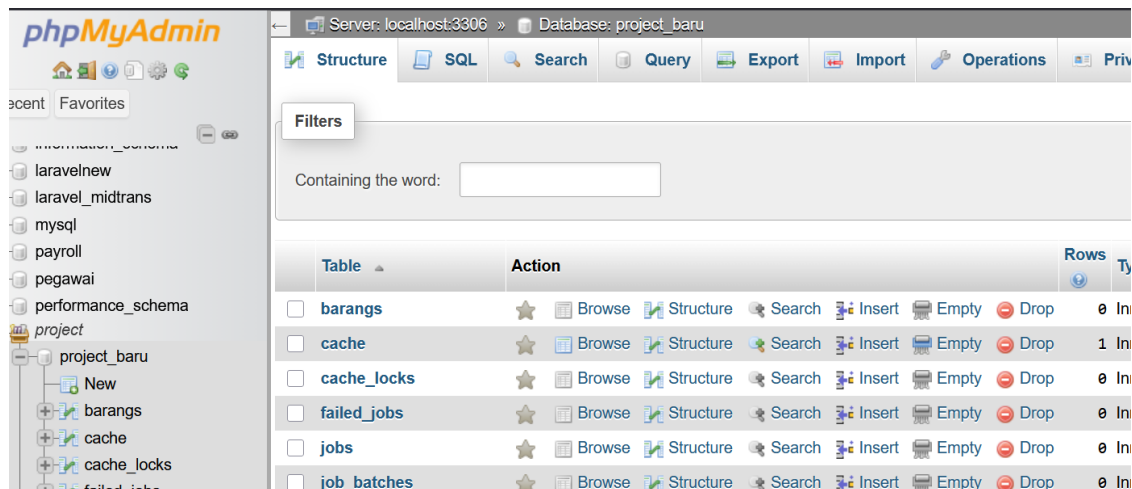
```
LENOVO@LAPTOP-2U2H9JC1 MINGW64 ~/project_baru
$ php artisan migrate

INFO: Running migrations.

2026_01_13_200331_ubah_tabel_produk ..... 65.66ms DONE
```

4. Cek Perubahan

Setelah melakukan migrate, kita bias melihat perubahan nama table, caranya dengan melihat nama database nya apakah sudah berganti Namanya, nama awal adalah produk menjadi barangs.



Gambar tersebut menunjukan bahwa nama table sudah berganti nama menjadi barangs, yang awalnya produk.

o Membuat Model

Setelah struktur database siap, langkah selanjutnya adalah membuat Model. Dalam pemahaman saya, Model adalah bagian dari konsep MVC (*Model-View-Controller*) yang bertugas menangani interaksi dengan database. Karena tabel saya bernama `barangs` (jamak), maka saya membuat model dengan nama `barang` (tunggal) agar Laravel dapat menghubungkannya secara otomatis tanpa konfigurasi tambahan.

Berikut adalah langkah-langkah yang saya kerjakan:

1. Men-generate File Model

Untuk membuat model, saya tidak membuatnya secara manual, melainkan memanfaatkan fitur Artisan agar lebih cepat dan standar. Saya membuka terminal di VSCode dan menjalankan perintah berikut:

```
LENOVO@LAPTOP-2U2H9JC1 MINGW64 ~/project_baru
$ php artisan make:model barang

INFO Model [C:\Users\LENOVO\project_baru\app\Models\barang.php] created successfully.

LENOVO@LAPTOP-2U2H9JC1 MINGW64 ~/project_baru
$
```

2. Verifikasi File

Saya kemudian mengecek direktori proyek untuk memastikan file tersebut benar-benar ada. Saya membuka folder `app > Models`, dan di sana saya menemukan file baru bernama `barang.php`. Ini membuktikan bahwa perintah sebelumnya telah berjalan dengan baik .

The file explorer shows the following structure:

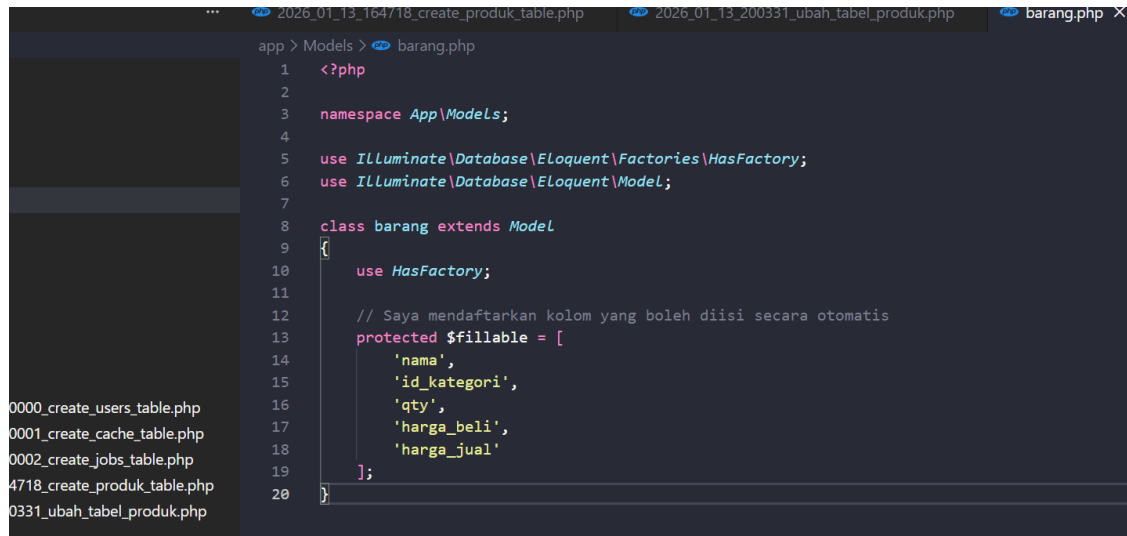
- PROJECT_BARU
 - .qodo
 - .vscode
 - app
 - Http\Controllers
 - Models
 - barang.php
 - User.php
 - Providers
 - bootstrap
 - config

The code editor shows a migration file with the following content:

```
database > migrations > 2026_01_13_200331_ubah_
8 {
12     public function up(): void
13     {
14         if (Schema::hasTable('produk')) {
15             Schema::rename('produk', 'barang');
16         }
17     }
18 }
19
20 /**
21  * Reverse the migrations.
22  */
23 public function down(): void
24 {
25     Schema::rename('barang', 'produk');
26 }
```

3. Konfigurasi Mass Assignment (\$fillable)

Langkah terakhir dan yang paling penting adalah mengonfigurasi isi file model tersebut. Saya membuka file `barang.php` dan menambahkan properti `$fillable`. Tujuan saya menambahkan kode ini adalah untuk mendefinisikan kolom-kolom mana saja yang diizinkan untuk diisi datanya secara massal (*mass assignment*) melalui perintah kode nanti (seperti saat menggunakan Seeder). Saya menuliskan skrip berikut di dalam class `barang`:



```
1  <?php
2
3  namespace App\Models;
4
5  use Illuminate\Database\Eloquent\Factories\HasFactory;
6  use Illuminate\Database\Eloquent\Model;
7
8  class barang extends Model
9  {
10     use HasFactory;
11
12     // Saya mendaftarkan kolom yang boleh diisi secara otomatis
13     protected $fillable = [
14         'nama',
15         'id_kategori',
16         'qty',
17         'harga_beli',
18         'harga_jual'
19     ];
20 }
```

Okee code tersebut bertujuan untuk memberikan akses, kolom mana saja yang boleh di isi, jadi ketika kolom tersebut tidak ada, maka tidak bisa di isi yang dapat menyebabkan error.

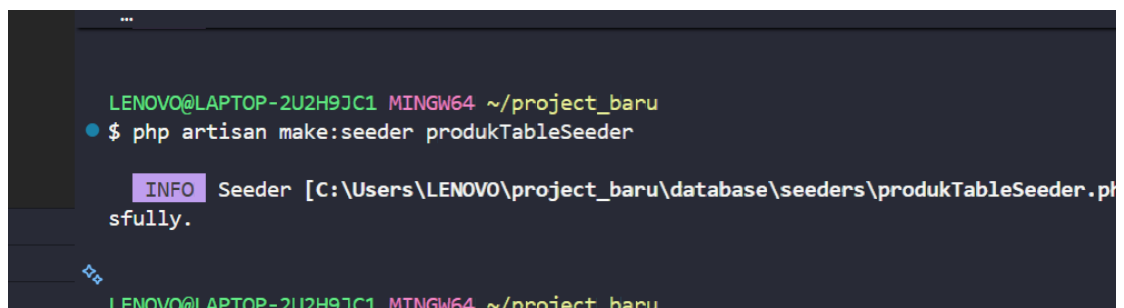
- Membuat Seeder

Setelah Model dan Tabel siap, saya membutuhkan data contoh untuk memastikan aplikasi berjalan dengan baik. Daripada menginput data satu per satu secara manual di database yang memakan waktu, saya memanfaatkan fitur **Seeder** untuk memasukkan data awal (*dummy data*) secara otomatis lewat kode.

Berikut adalah tahapan kerja yang saya lakukan:

1. Membuat File Seeder

Langkah pertama, saya membuat file seeder baru melalui terminal. Saya menjalankan perintah Artisan berikut:



```
LENOVO@LAPTOP-2U2H9JC1 MINGW64 ~/project_baru
$ php artisan make:seeder produkTableSeeder

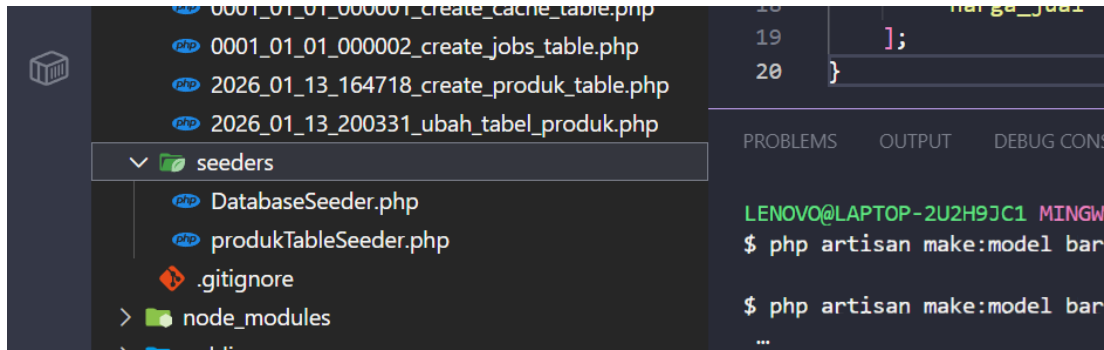
INFO Seeder [C:\Users\LENOVO\project_baru\database\seeders\produkTableSeeder.php] created successfully.

LENOVO@LAPTOP-2U2H9JC1 MINGW64 ~/project_baru
```

Terminal kemudian memberikan respon *INFO Seeder created successfully*, yang berarti file tersebut berhasil dibuat .

2. Pengecekan File

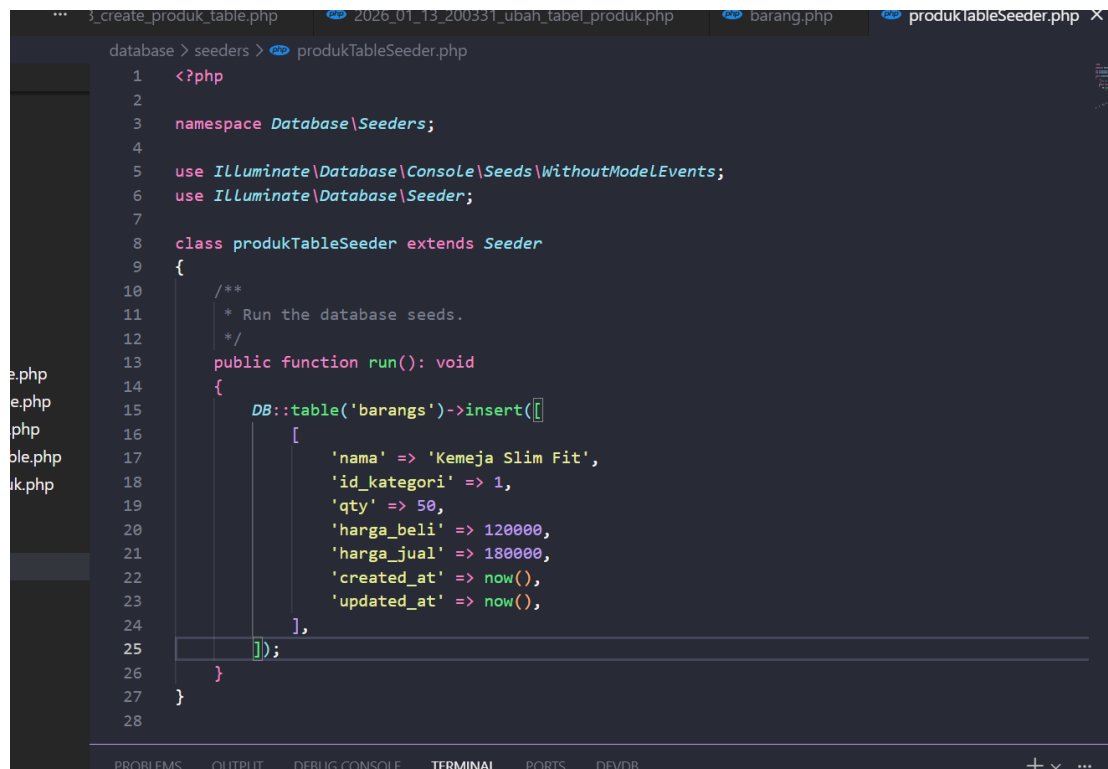
Saya memastikan keberadaan file tersebut dengan membuka direktori database/seeders di VSCode. Di sana, saya melihat file baru bernama produkTableSeeder.php sudah tersedia.



3. Menulis Logika Input Data

Selanjutnya, saya membuka file `produkTableSeeder.php` tersebut untuk memasukkan data yang ingin saya simpan. Pada method `run()`, saya menggunakan fungsi `DB::table` yang diarahkan ke tabel `barangs`. Saya memasukkan beberapa data produk contoh seperti "Kemeja Slim Fit", "Kaos Polos Premium", dan "Jaket Kulit" lengkap dengan detail harga dan stoknya.

Kode yang saya tuliskan kurang lebih sebagai berikut:



4. Mendaftarkan Seeder Utama

Agar file seeder yang baru saya buat bisa dijalankan secara global, saya perlu mendaftarkannya terlebih dahulu. Saya membuka file utama `DatabaseSeeder.php` (masih di folder yang sama).

Di dalam method `run()`, saya menambahkan perintah pemanggilan kelas:

```

1  <?php
2
3  namespace Database\Seeders;
4
5  use App\Models\User;
6  use Illuminate\Database\Console\Seeds\WithoutModelEvents;
7  use Illuminate\Database\Seeder;
8
9  class DatabaseSeeder extends Seeder
10 {
11     use WithoutModelEvents;
12
13     /**
14      * Seed the application's database.
15      */
16     public function run(): void
17     {
18         // User::factory(10)->create();
19         $this->call(produkTableSeeder::class);
20
21         User::factory()->create([
22             'name' => 'Test User',
23             'email' => 'test@example.com',
24         ]);
25     }
26 }
27

```

5. Mengeksekusi Seeder

Setelah semua kode tersimpan rapi, saya kembali ke terminal untuk menjalankan proses pengisian data. Saya mengetikkan perintah:

```

LENOVO@LAPTOP-2U2H9JC1 MINGW64 ~/project_baru
$ php artisan db:seed

INFO Seeding database.

Database\Seeders\produkTableSeeder ..... RUNNING
Database\Seeders\produkTableSeeder ..... 109 ms DONE

```

6. Verifikasi Akhir

Untuk membuktikan bahwa data benar-benar masuk, saya membuka browser dan mengakses **phpMyAdmin**. Saat saya mengecek tabel **barang**s, data-data yang saya tulis di kode tadi (Kemeja, Kaos, Jaket) sudah tersimpan rapi di dalam tabel. Ini menandakan fitur Seeder telah berfungsi dengan sempurna.

phpMyAdmin

Home

Database

SQL

Search

Insert

Export

Import

Privileges

Operations

Triggers

Favorite

Server: localhost:3306 » Database: project_baru » Table: barangs

Browse

Structure

SQL

Search

Insert

Export

Import

Privileges

Operations

Triggers

Showing rows 0 - 0 (1 total, Query took 0.0005 seconds.)

SELECT * FROM `barangs`

☐ Profiling

[\[Edit inline \]](#)

[\[Edit \]](#)

[\[Explain SQL \]](#)

[\[Create PHP code \]](#)

[\[Refresh \]](#)

☐ Show all

Number of rows:

25

Filter rows:

Search this table

Extra options

id

nama

id_kategori

qty

harga_beli

harga_jual

created_at

updated_at

☐

Edit

Copy

Delete

1

Kemeja Slim Fit

1

50

120000

180000

2026-01-13 20:30:39

2026-01-13 20:30:39

Check all

With selected:

Edit

Copy

Delete

Export

Data sudah masuk ke dalam database, menandakan berhasil, itulah beberapa fitur yang ada di dalam Laravel, sebenarnya masih banyak hanya itulah beberapa fitur yang bisa digunakan di dalam Laravel.